



CS1002 – Programming Fundamentals

Lecture # 13
Thursday, October 13, 2022
FALL 2022
FAST – NUCES, Faisalabad Campus

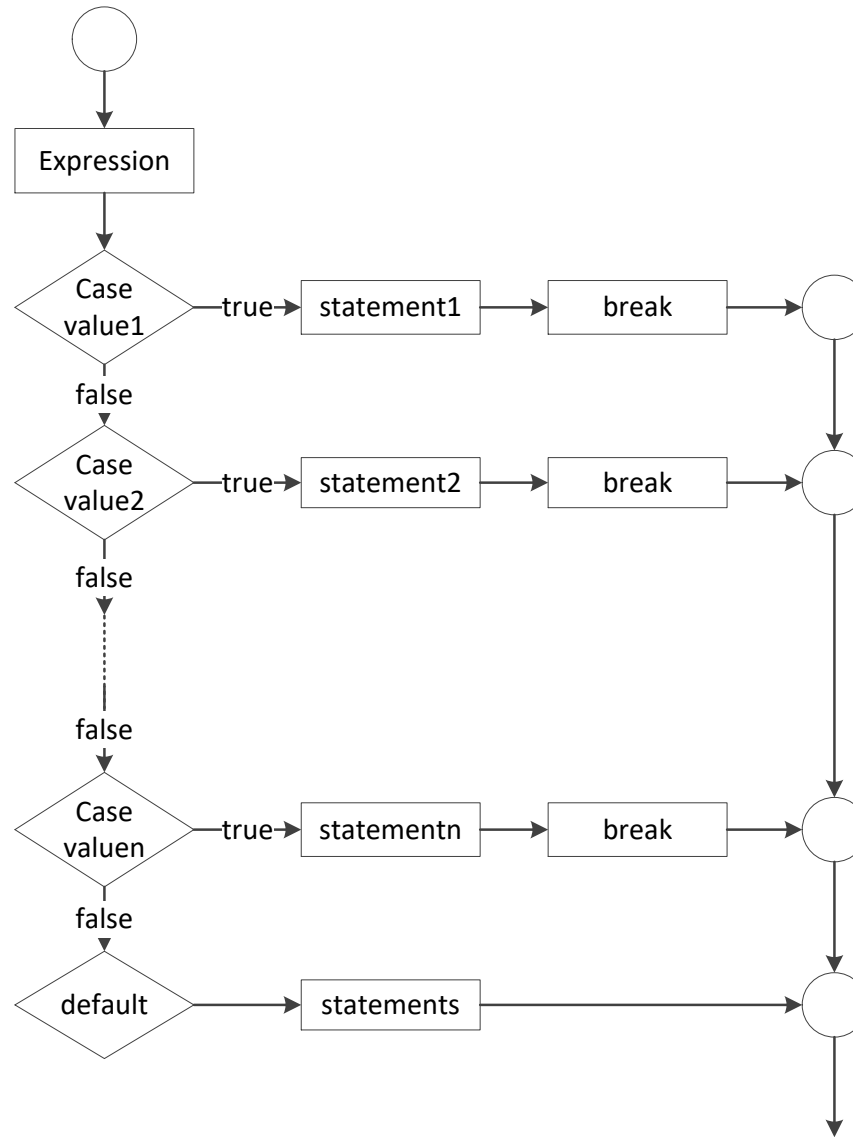
Rizwan Ul Haq

switch Structures

- **switch structure:** Alternate to **if-else**
- switch **(integral)** expression is evaluated first
- Value of the expression determines which corresponding action is taken
- Expression is sometimes called the selector

```
switch (expression)
{
    case value1:
        statements1
        break;
    case value2:
        statements2
        break;
    .
    .
    .
    case valuen:
        statementsn
        break;
    default:
        statements
}
```

switch Structures (cont'd.)



switch Structures (cont'd.)

- One or more statements may follow a case label
- **Braces are not needed to turn multiple** statements into a single compound statement
- The break statement may or may not appear after each statement
- **switch, case, break,** and **default** are reserved words

EXAMPLE 4-21

Consider the following statements, in which `grade` is a variable of type `char`.

```
switch (grade)
{
case 'A':
    cout << "The grade point is 4.0.";
    break;
case 'B':
    cout << "The grade point is 3.0.";
    break;
case 'C':
    cout << "The grade point is 2.0.";
    break;
case 'D':
    cout << "The grade point is 1.0.";
    break;
case 'F':
    cout << "The grade point is 0.0.";
    break;
default:
    cout << "The grade is invalid.";
}
```

In this example, the expression in the `switch` statement is a variable identifier. The variable `grade` is of type `char`, which is an integral type. The possible values of `grade` are 'A', 'B', 'C', 'D', and 'F'. Each `case` label specifies a different action to take, depending on the value of `grade`. If the value of `grade` is 'A', the output is:

The grade point is 4.0.

```
int main() //Line 3
{ //Line 4
    int testscore; //Line 5

    std::cout << "Enter the test score: "; //Line 6
    std::cin >> testscore; //Line 7
    std::cout << std::endl; //Line 8
    switch (testscore / 10) //Line 9
    { //Line 10
    case 0: //Line 11
    case 1: //Line 12
    case 2: //Line 13
    case 3: //Line 14
    case 4: //Line 15
    case 5: //Line 16
        std::cout << "The grade is F." << std::endl; //Line 17
        break; //Line 18
    case 6: //Line 19
        std::cout << "The grade is D." << std::endl; //Line 20
        break; //Line 21
    case 7: //Line 22
        std::cout << "The grade is C." << std::endl; //Line 23
        break; //Line 24
    case 8: //Line 25
        std::cout << "The grade is B." << std::endl; //Line 26
        break; //Line 27
    case 9: //Line 28
    case 10: //Line 29
        std::cout << "The grade is A." << std::endl; //Line 30
        break; //Line 31
    default: //Line 32
        std::cout << "Incorrect marks" << std::endl; //Line 33
    } //Line 34
    return 0; //Line 35
} //Line 36
```

Output = ?

```
#include<iostream>
using namespace std;
int main() {
    int number;
    cout << "Enter a number in the range 0 - 7 : ";
    cin >> number;
    cout << "The number you entered is = " << number << endl;
    switch (number) {
    case 0:
    case 1:
        cout << "Learning to use ";
    case 2:
        cout << "C++'s ";
    case 3:
        cout << "switch structure." << endl;
        break;
    case 4:
        break;
    case 5:
        cout << "This program shows the effect ";
    case 6:
    case 7:
        cout << "of break statement." << endl;
        break;
    default:
        cout << "The number is out of range." << endl;
    }
    cout << "Out of the switch structure" << endl;
    return 0;
}
```

0
2
4
5
7
8

Avoiding Bugs by Avoiding Partially Understood Concepts and Techniques: Revisited

- To output results correctly
- The **switch** structure must include a **break** statement after each **cout** statement

Terminating a Program with the `assert` Function

- Certain types of errors that are very difficult to catch can occur in a program
 - **Example:** Division by zero can be difficult to catch using any of the programming techniques examined so far
- The predefined function, **`assert`**, is useful in stopping program execution when certain elusive errors occur

The assert Function (cont'd.)

➤ Syntax:

```
assert (expression);
```

- **expression** is any logical expression
- If **expression** evaluates to **true**, the next statement executes
- If **expression** evaluates to **false**, the program terminates and indicates where in the program the error occurred
- To use **assert**, include **cassert** header file

The assert Function (cont'd.)

- **assert** is useful for enforcing programming constraints during program development
- After developing and testing a program, remove or disable assert statements
- The preprocessor directive **#define NDEBUG** must be placed before the directive **#include <cassert>** to disable the assert statement

assert example

```
#include <iostream>
//#define NDEBUG
#include <cassert>

using namespace std;

int main()
{
    int den, num;
    cout << "Enter two integers" << endl;
    cin >> num >> den;
    assert(den != 0);
    cout << "Moving forward" << endl;
    cout << "num / den = " << num / den << endl;
    return 0;
}
```

Complete Example for Practice

Programming Example: Cable Company Billing

- This programming example calculates a customer's bill for a local cable company
- There are two types of customers:
 - Residential
 - Business
- Two rates for calculating a cable bill:
 - One for residential customers
 - One for business customers

Programming Example: Rates

- For residential customer:
 - Bill processing fee: \$4.50
 - Basic service fee: \$20.50
 - Premium channel: \$7.50 per channel
- For business customer:
 - Bill processing fee: \$15.00
 - Basic service fee: \$75.00 for first 10 connections/\$5.00 for each additional one
 - Premium channel cost: \$50.00 per channel for any number of connections

Programming Example: Requirements

- Ask user for account number and customer code
- Assume **R** or **r** stands for residential customer and **B** or **b** stands for business customer

Programming Example: Input and Output

➤ Input:

- Customer account number
- Customer code
- Number of premium channels
- For business customers, number of basic service connections

➤ Output:

- Customer's account number
- Billing amount

Programming Example: Program Analysis

- **Purpose:** calculate and print billing amount
- **Calculating billing amount requires:**
 - Customer for whom the billing amount is calculated (residential or business)
 - Number of premium channels to which the customer subscribes
- **For a business customer, you need:**
 - Number of basic service connections
 - Number of premium channels

Programming Example: Program Analysis (cont'd.)

- Data needed to calculate the bill, such as bill processing fees and the cost of a premium channel, are known quantities
- The program should print the billing amount to two decimal places

Programming Example: Algorithm Design

- Set precision to two decimal places
- Prompt user for account number and customer type
- If customer type is **R** or **r**
 - Prompt user for number of premium channels
 - Compute and print the bill
- If customer type is **B** or **b**
 - Prompt user for number of basic service connections and number of premium channels
 - Compute and print the bill

Programming Example: Variables and Named

Const

```
int accountNumber ;      //Variable to store the customer's
                          //Account number
char customerType ;      //Variable to store customer type
int numOfPremiumChannels ; //Variable to store the number of premium
                          //Channels to which the customer subscribed
int numOfBasicSerCon ;   //Variable to store the number of basic
                          //Channels to which the customer subscribed
double amountDue ;      //Variable to store the billing amount

//Named Constants - Residential customers
const double RES_BILL_PROC_FEE = 4.5 ;
const double RES_BASIC_SERV_COST = 20.5 ;
const double RES_COST_PREM_CHANNEL = 7.5 ;

//Named Constants - Business customers
const double BUS_BILL_PROC_FEE = 15.00 ;
const double BUS_BASIC_SERV_COST = 75.0 ;
const double BUS_BASIC_CONN_COST = 5.0 ;
const double BUS_COST_PREM_CHANNEL = 50.0 ;
```

Programming Example: Formulas

➡ Billing for residential customers:

```
amountDue = RES_BILL_PROC_FEES +  
             RES_BASIC_SERV_COST +  
             numOfPremChannels *  
             RES_COST_PREM_CHANNEL;
```

Programming Example: Formulas (cont'd.)

➤ Billing for business customers:

```
if (numOfBasicServConn <= 10)
    amountDue = BUS_BILL_PROC_FEES +
                BUS_BASIC_SERV_COST +
                numOfPremChannels *
                BUS_COST_PREM_CHANNEL ;
else
    amountDue = BUS_BILL_PROC_FEES +
                BUS_BASIC_SERV_COST +
                (numOfBasicServConn - 10) *
                BUS_BASIC_CONN_COST +
                numOfPremChannels *
                BUS_COST_PREM_CHANNEL ;
```

Programming Example: Main Algorithm

1. Output floating-point numbers in fixed decimal with decimal point and trailing zeros
 - Output floating-point numbers with two decimal places and set the precision to two decimal places
2. Prompt user to enter account number
3. Get customer account number
4. Prompt user to enter customer code
5. Get customer code

Programming Example: Main Algorithm (cont'd.)

6. If the customer code is **r** or **R**,
 - Prompt user to enter number of premium channels
 - Get the number of premium channels
 - Calculate the billing amount
 - Print account number and billing amount

Programming Example: Main Algorithm (cont'd.)

7. If customer code is **b** or **B**

- Prompt user to enter number of basic service connections
- Get number of basic service connections
- Prompt user to enter number of premium channels
- Get number of premium channels
- Calculate billing amount
- Print account number and billing amount

Programming Example: Main Algorithm

(cont'd.)

- If customer code is other than **r**, **R**, **b**, or **B**, output an error message

Another Programming Example

```
1 // Example
2 // Using if statements, relational
3 // operators, and equality operators.
4 #include <iostream>
5
6 using std::cout; // program uses cout
7 using std::cin; // program uses cin
8 using std::endl; // program uses endl
9
10 // function main begins program execution
11 int main()
12 {
13     int num1; // first number to be read from user
14     int num2; // second number to be read from user
15
16     cout << "Enter two integers, and I will tell you\n"
17         << "the relationships they satisfy: ";
```

```
18  cin >> num1 >> num2; // read two integers
19
20  if ( num1 == num2 )
21      cout << num1 << " is equal to " << num2 << endl;
22
23  if ( num1 != num2 )
24      cout << num1 << " is not equal to " << num2 << endl;
25
26  if ( num1 < num2 )
27      cout << num1 << " is less than " << num2 << endl;
28
29  if ( num1 > num2 )
30      cout << num1 << " is greater than " << num2 << endl;
31
32  if ( num1 <= num2 )
33      cout << num1 << " is less than or equal to "
34          << num2 << endl;
35
36  if ( num1 >= num2 )
37      cout << num1 << " is greater than or equal to "
38          << num2 << endl;
39
40  return 0; // indicate that program ended successfully
41
42  } // end function main
```

Sample Run 1

Enter two integers, and I will tell you
the relationships they satisfy: 35 17

35 is not equal to 17

35 is greater than 17

35 is greater than or equal to 17

Sample Run 2

Enter two integers, and I will tell you
the relationships they satisfy: 10 10

10 is equal to 10

10 is less than or equal to 10

10 is greater than or equal to 10

Summary

- Control structures alter normal control flow
- Most common control structures are selection and repetition
- Relational operators: `==`, `<`, `<=`, `>`, `>=`, `!=`
- Logical expressions evaluate to 1 (true) or 0 (false)
- Logical operators: `!` (not), `&&` (and), `||` (or)

Summary (cont'd.)

- Two selection structures: one-way selection and two-way selection
- The expression in an **if** or **if...else** structure is usually a logical expression
- No stand-alone **else** statement in C++
 - Every **else** has a related **if**
- A sequence of statements enclosed between braces, { and }, is called a compound statement or block of statements

Summary (cont'd.)

- Using assignment in place of the equality operator creates a semantic error
- **switch** structure handles multiway selection
- **break** statement ends **switch** statement
- Use **assert** to terminate a program if certain conditions are not met

Questions

